



JAVASCRIPT y HTML

Contenido



- ❖ Formularios y elementos asociados.
- ❖ DOM.
- ❖ Selectores.
- ❖ Eventos.
- ❖ Ventanas.



Formularios y elementos asociados (1/7)

Etiquetas	Descripción
<form>	Define un formulario HTML para entradas de usuario.
<input>	Define un control de entrada.
<textarea>	Define un control de entrada multilínea.
<button>	Define un botón 'clicable'.
<select>	Define una lista desplegable (combo box).
<optgroup>	Define un grupo de opciones para una lista desplegable.
<option>	Define una opción para una lista desplegable.
<label>	Define una etiqueta para los controles <input>.
<datalist> *	Especifica una lista de opciones predefinidas para inputs.
<keygen> *	Define un campo generador de clave (para <form>).
<output> *	Define el resultado de un cálculo.



Formularios y elementos asociados (2/7)

Los formularios permiten, desde dentro de una aplicación Web, solicitar información al visitante.

Estos formularios estarán compuestos por tantos campos como informaciones deseamos obtener.



Formularios y elementos asociados (3/7)

El tag `<input>` es la segunda etiqueta más utilizada en formularios.

Es un tag multifunción, ya que de acuerdo al valor del atributo **type**, se podrán definir: textbox, password, checkbox, radiobutton, etc.



Formularios y elementos asociados (4/7)

Input Types	Input Attributes
color	autocomplete
date	autofocus
datetime	form
email	height / width
month	list
number	min / max
range	multiple
search	pattern (expresiones regulares)
tel	placeholder
time	required
url	step
week	



Formularios y elementos asociados (5/7)

Caja de texto multilínea

- ❖ El atributo **rows** especifica el número de filas que se visualizará en el área de texto.
- ❖ Mientras que el atributo **cols** indica la cantidad de caracteres por fila.
- ❖ El atributo **wrap** indica que se saltará automáticamente a la línea siguiente cuando se complete la línea en la que se escribe.



Formularios y elementos asociados (6/7)

La etiqueta que permite crear una lista desplegable es **<select>**.

- ❖ Las opciones se especifican con el tag **<option>**.
- ❖ El atributo **value** indica el valor asociado con la opción especificada. Si se omite, toma el valor del texto.
- ❖ El atributo **size** indica la cantidad de opciones que se visualizarán en la lista.
- ❖ Para agrupar opciones se utiliza el tag **<optgroup>**.
- ❖ Cada grupo quedará identificado con el atributo **label**.



Formularios y elementos asociados (7/7)

- ❖ La etiqueta **<datalist>** especifica una lista predefinida de opciones para un **<input>**.
- ❖ Las opciones se especifican con el tag **<option>**.
- ❖ El atributo **value** indica el valor asociado con la opción especificada.
- ❖ El atributo **list** del **<input>** debe referir al atributo **id** del **<datalist>**.



Ejemplos



DOM (Document Object Model)

- ❖ Mediante el DOM se puede acceder al contenido y estilo de cada **tag** del documento y modificarlo de acuerdo a algún evento.
- ❖ También se puede insertar, borrar, modificar etiquetas de HTML.
- ❖ Todos esto sin tener que recargar la página del servidor, es decir, todo se hace en el cliente (navegador) mediante **JavaScript**.



DOM (1/3)

Árbol de nodos

- ❖ DOM transforma todos los documentos HTML en un conjunto de **nodos**, que están interconectados y que representan los contenidos de las páginas Web y las relaciones entre ellos.



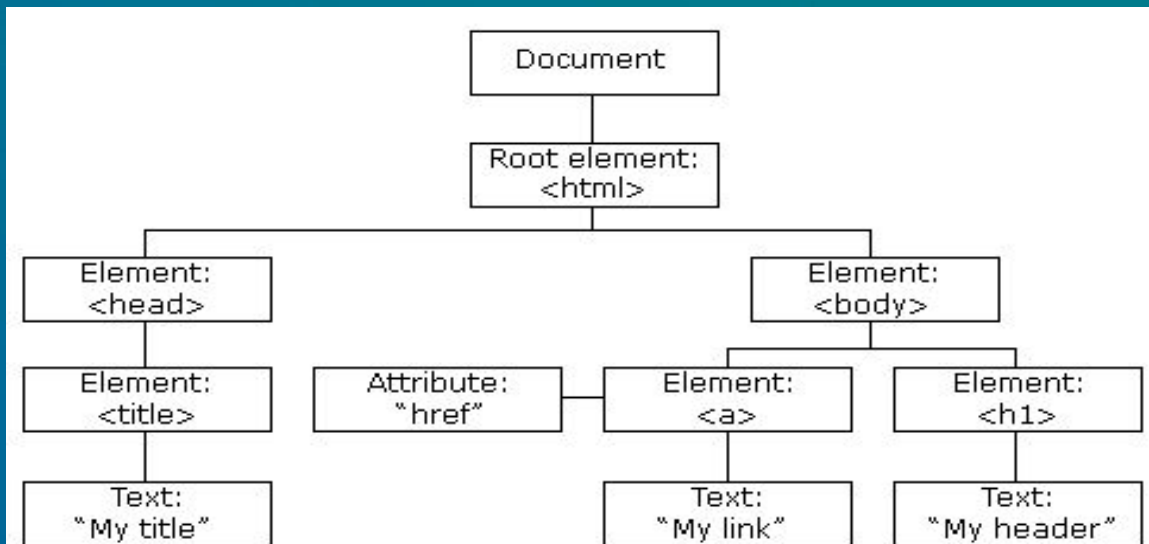
DOM (2/3)

```
<!DOCTYPE html>
<html>
<head>
  <title>My title</title>
</head>
<body>

<a href="">My link</a>

<h1>My header</h1>

</body>
</html>
```





DOM (3/3)

DOM proporciona dos métodos alternativos para acceder a un nodo específico:

- ❖ acceso a través de sus **nodos padre**.
- ❖ acceso directo.

Para acceder a los elementos del DOM, usaremos ***selectores***.

Cada selector puede retornar un solo elemento o una lista de elementos.

Para poder hacer uso de los selectores debemos hacer uso del objeto ***document*** ya que los selectores son métodos del mismo.



Selectores (1/2)

- ❖ **querySelector()**: Obtiene el primer elemento del DOM según el parámetro recibido.
- ❖ **querySelectorAll()**: Obtiene todos los elementos del DOM según el parámetro recibido.

Nota:

*Si el parámetro recibido comienza con el caracter . (punto) seguido de un identificador, se buscará el o los elementos de HTML cuyo atributo **class** coincida.*

*Si al parámetro se le antepone el caracter #, se realizará la búsqueda por el valor del atributo **id** del elemento.*

Si se coloca el nombre de una etiqueta HTML, seleccionará al primero o a todos los elementos de ese tipo dentro del documento.



Selectores (2/2)

- ❖ **getElementsByTagName():** Obtiene todos los elementos de la página cuya **etiqueta** sea igual que el parámetro que se le pasa a la función.
- ❖ **getElementsByTagName():** Es similar a la anterior, pero en este caso se buscan los elementos cuyo atributo **name** sea igual al parámetro proporcionado.
- ❖ **getElementById():** Este selector recibe una cadena con el valor del **id** del elemento que se está buscando.



Ejemplos



Manipulando el DOM (1/3)

Mediante DOM, es posible acceder de forma sencilla a todos los atributos HTML y todas las propiedades CSS de cualquier elemento de la página.

Los atributos de los elementos de la página se transforman automáticamente en propiedades de los nodos.

- ❖ **innerHTML**
Permite leer o modificar el *contenido* de una etiqueta HTML.
- ❖ **innerText**
Permit leer o modificar el *texto* de una etiqueta HTML.



Manipulando el DOM (2/3)

Para obtener el valor de cualquier propiedad CSS del nodo, se debe utilizar el atributo **style**.

La transformación del nombre de las propiedades CSS compuestas consiste en eliminar todos los guiones medios (-) y escribir en mayúscula la letra siguiente a cada guión medio.

DOM utiliza el nombre **className** para acceder al atributo **class** del elemento.



Manipulando el DOM (3/3)

JavaScript brinda una propiedad y varios métodos que permiten hacer diversas acciones con el atributo class de un elemento.

- ❖ **classList.add()**
Permite agregar al elemento seleccionado, una clase nueva.
- ❖ **classList.remove()**
Permite quitarle una clase existente al elemento seleccionado.
- ❖ **classList.toggle()**
Si existe una clase en el elemento seleccionado, la remueve, de lo contrario, la agrega.
- ❖ **classList.contains()**
Devuelve un valor booleano que indica si un elemento tiene una clase determinada.



Ejemplos



Eventos (1/3)

Un evento es algo que ocurre en el navegador o que es ejecutado por alguna acción del usuario.

Los eventos hacen posible que los usuarios transmitan información a los programas.

JavaScript define numerosos eventos que permiten una interacción completa entre el usuario y las páginas/aplicaciones Web.

JavaScript permite asignar una función a cada uno de los eventos.

De esta forma, cuando se produce cualquier evento, JavaScript ejecuta su función asociada.

Este tipo de funciones se denominan "event handlers" (manejadores de eventos).



Eventos (2/3)

Evento	Descripción	Elementos
onblur	Deseleccionar el elemento.	button, input, label, select, textarea, body
onchange	Cambio el elemento.	input, select, textarea
onclick	Clickear y soltar el mouse.	Todos los elementos
ondblclick	Clickear dos veces seguidas.	Todos los elementos
onfocus	Seleccionar un elemento.	button, input, label, select, textarea, body
onkeydown	Pulsar una tecla (sin soltarla).	Eventos de formulario y body
onkeypress	Pulsar una tecla.	Eventos de formulario y body
onkeyup	Soltar una tecla pulsada.	Eventos de formulario y body
onload	La página se ha cargado por completo.	body



Eventos (3/3)

Evento	Descripción	Elementos
onmousedown	Pulsar, sin soltar, un botón del mouse.	Todos los elementos
onmousemove	Mover el mouse.	Todos los elementos
onmouseout	El mouse sale del elemento.	Todos los elementos
onmouseover	El mouse entra en el elemento.	Todos los elementos
onmouseup	Soltar el botón que estaba pulsado.	Todos los elementos
onreset	Borrar los datos de un formulario.	form
onresize	Se modifica el tamaño de la ventana del navegador.	body
onsubmit	Enviar el formulario.	form
onunload	Se abandona la página.	body



Manejadores de eventos (1/3)

Manejadores cómo atributos:

- ❖ Se trata del método más sencillo de indicar el código JavaScript que se debe ejecutar cuando se produzca un evento determinado.
- ❖ En este caso, el código se incluye en un atributo del propio elemento HTML.

```
<input type="button" value="Click!" onclick="console.log('click');">
```



Manejadores de eventos (2/3)

Manejadores cómo funciones:

- ❖ Esta técnica consiste en extraer todas las instrucciones de JavaScript y agruparlas en una función externa.
- ❖ En el atributo del elemento HTML se incluye el nombre de la función.
- ❖ La llamada a la función se realiza de la forma habitual, indicando su nombre seguido de los paréntesis y de forma opcional, incluyendo todos los parámetros que se necesiten.

```
<input type="button" value="Click!" onclick="mostrarMensaje()">
```



Manejadores de eventos (3/3)

Manejadores semánticos:

Esta técnica es una evolución del método de las funciones externas, ya que se basa en utilizar las propiedades DOM de los elementos HTML para asignar todas las funciones externas que actúan de manejadores de eventos.

Dicha técnica consiste en:

- ❖ Crear una función de JavaScript encargada de manejar el evento.
- ❖ Asignar la función externa al evento correspondiente en el elemento deseado, de forma dinámica.



Ejemplos



Ventanas

alert():

Es la ventana más utilizada. Sirve para mostrar mensajes literales y/o valores de variables al cliente de una aplicación Web.

prompt():

Presentan una ventana con un mensaje y un cuadro de texto donde el usuario introduce aquello que se le pide.

La cadena tecleada por el usuario puede almacenarse en una variable para su futuro uso.

confirm():

Presenta una ventana con un mensaje y dos botones, Aceptar y Cancelar, que el usuario pulsará en función del contenido del mensaje.

Si el usuario pulsa Aceptar, confirm devuelve un valor Verdadero (true), mientras que si pulsa Cancelar, devuelve un valor Falso (false).



Ejemplos



Ejercitación